
بهینه‌سازی داده‌های سری-زمانی در PostgreSQL

هدف از این کارگاه، بررسی عملی و مقایسه عملکرد دو رویکرد متفاوت برای ذخیره‌سازی حجم بالای داده‌های سری-زمانی (Time-Series) در پایگاه داده PostgreSQL بود. در این پروژه، ما عملکرد یک جدول بزرگ و واحد را با یک جدول پارتیشن‌بندی شده بر اساس ماه مقایسه کردیم. معیارهای اصلی برای این مقایسه، زمان اجرای کوئری (Execution Time) و میزان خواندن داده از دیسک (I/O) بودند که با استفاده از دستور EXPLAIN (ANALYZE, BUFFERS) اندازه‌گیری شدند.

سناریوی اول: استفاده از یک جدول بزرگ (Baseline)

در این سناریو، تمام ۱۰ میلیون رکورد داده سنسور در یک جدول واحد به نام `sensor_data` ذخیره شدند. یک ایندکس B-Tree نیز روی ستون `reading_time` برای بهبود سرعت جستجوها ایجاد شد.

نتایج تحلیل کوثری‌ها

کوثری ۱: خواندن داده‌های یک روز خاص (۳۱ دسامبر ۲۰۲۳)

- زمان اجرا : ~ 1748 میلی‌ثانیه (حدود ۱.۷ ثانیه)
- بلاک‌های خوانده شده از دیسک: 22,736
- تحلیل : با وجود استفاده از ایندکس، پایگاه داده مجبور شد برای یافتن داده‌های یک روز، بیش از ۲۲ هزار بلاک را از دیسک بخواند. این حجم بالای I/O باعث کندی شدید کوثری شد و نشان داد که در حالت "کش سرد (Cold Cache)"، عملکرد جدول بزرگ بسیار ضعیف است.

QUERY PLAN	
text	
1	Bitmap Heap Scan on sensor_data (cost=1240.93..71503.18 rows=50000 width=32) (actual time=7.718..1742.035 rows=27150.00 loops=1)
2	Recheck Cond: ((reading_time >= '2023-12-31 00:00:00':timestamp without time zone) AND (reading_time < '2024-01-01 00:00:00':timestamp without time ...
3	Heap Blocks: exact=22731
4	Buffers: shared hit=96 read=22736 dirtied=22674 written=15449
5	-> Bitmap Index Scan on idx_sensor_time (cost=0.00..1228.43 rows=50000 width=0) (actual time=4.413..4.414 rows=27150.00 loops=1)
6	Index Cond: ((reading_time >= '2023-12-31 00:00:00':timestamp without time zone) AND (reading_time < '2024-01-01 00:00:00':timestamp without time ...
7	Index Searches: 1
8	Buffers: shared hit=39 read=62
9	Planning:
10	Buffers: shared hit=10 read=6 dirtied=3
11	Planning Time: 0.774 ms
12	Execution Time: 1748.062 ms

Total rows: 12 Query complete 00:00:01.790

کوثری ۲: محاسبه میانگین روزانه دما برای یک ماه (مارس ۲۰۲۳)

- زمان اجرا: ~2199 میلی ثانیه (حدود ۲.۲ ثانیه)
- بلاک‌های خوانده شده از دیسک: 76,415
- تحلیل: این کوثری تحلیلی نیز به دلیل نیاز به اسکن حجم عظیمی از داده‌ها بسیار کند عمل کرد. خواندن بیش از ۷۶ هزار بلاک از دیسک، گلوگاه اصلی عملکرد بود و ناکارآمدی ساختار تک‌جدولی را برای تحلیل‌های زمانی اثبات کرد.

challenge.pdf		
pgAdmin 4		
http://localhost:5050/browser/		
chatgpt.com Google Gemini Translate Gmail YouTube آکادمی همراه اول background music All Bookmarks		
QUERY PLAN		
text		
6	Recheck Cond: ((reading_time >= '2023-03-01 00:00:00':timestamp without time zone) AND (reading_time < '2023-04-01 00:00:00':timestamp without time ...	
7	Rows Removed by Index Recheck: 4109450	
8	Heap Blocks: exact=40444 lossy=33085	
9	Buffers: shared hit=2 read=76415 written=12141	
10	-> Bitmap Index Scan on idx_sensor_time (cost=0.00..21268.96 rows=868452 width=0) (actual time=142.391..142.391 rows=849706.00 loops=1)	
11	Index Cond: ((reading_time >= '2023-03-01 00:00:00':timestamp without time zone) AND (reading_time < '2023-04-01 00:00:00':timestamp without time ...	
12	Index Searches: 1	
13	Buffers: shared hit=2 read=2886 written=800	
14	Planning:	
15	Buffers: shared hit=23	
16	Planning Time: 0.416 ms	
17	JIT:	
18	Functions: 11	
19	Options: Inlining false, Optimization false, Expressions true, Deforming true	
20	Timing: Generation 1.288 ms (Deform 0.426 ms), Inlining 0.000 ms, Optimization 1.601 ms, Emission 25.126 ms, Total 28.015 ms	
21	Execution Time: 2198.923 ms	
Total rows: 21	Query complete 00:00:02.322	

کوثری ۳: خواندن داده‌های یک دستگاه خاص در یک ماه (دستگاه ۴۲، ماه مارس)

- زمان اجرا : ~1538 میلی‌ثانیه (حدود ۱.۵ ثانیه)
- بلاک‌های خوانده شده از دیسک : 76,420
- تحلیل: در این حالت نیز پایگاه داده مجبور شد تمام داده‌های ماه مارس را از دیسک بخواند و سپس آن‌ها را فیلتر کند. این کار به دلیل عدم وجود ایندکس مناسب روی device_id بسیار غیربهرینه و کند بود.

10	Heap Blocks: exact=17655 lossy=14408
11	Buffers: shared hit=37 read=76420
12	Worker 0: Heap Blocks: exact=11827 lossy=9849
13	Worker 1: Heap Blocks: exact=10962 lossy=8828
14	-> Bitmap Index Scan on idx_sensor_time (cost=0.00..21268.96 rows=868452 width=0) (actual time=377.930..377.931 rows=849706.00 loops=1)
15	Index Cond: ((reading_time >= '2023-03-01 00:00:00'::timestamp without time zone) AND (reading_time < '2023-04-01 00:00:00'::timestamp without time ...
16	Index Searches: 1
17	Buffers: shared read=2888
18	Planning:
19	Buffers: shared hit=2 read=4 dirtied=2
20	Planning Time: 0.348 ms
21	JIT:
22	Functions: 12
23	Options: Inlining false, Optimization false, Expressions true, Deforming true
24	Timing: Generation 2.533 ms (Deform 0.422 ms), Inlining 0.000 ms, Optimization 1.375 ms, Emission 43.027 ms, Total 46.935 ms
25	Execution Time: 1538.366 ms
Total rows: 25 Query complete 00:00:01.868	

سناریوی دوم: استفاده از جدول پارتیشن‌بندی شده

در این رویکرد، جدول `sensor_data_p` بر اساس بازه زمانی (RANGE) روی ستون `reading_time` پارتیشن‌بندی شد. برای هر ماه از سال ۲۰۲۳، یک پارتیشن (جدول فرزند) مجزا ایجاد گردید.

نتایج تحلیل کوئری‌ها

کوئری ۱: خواندن داده‌های یک روز خاص (۳۱ دسامبر ۲۰۲۳)

- زمان اجرا: ~44 میلی‌ثانیه
- بلاک‌های خوانده شده از دیسک: 5,965
- تحلیل: عملکرد حدود ۴۰ برابر سریع‌تر شد. پلنر کوئری از قابلیت کلیدی `Partition Pruning` (هرس کردن پارتیشن) استفاده کرد و ۱۱ پارتیشن غیرمرتبط را به طور کامل نادیده گرفت.

Data Output		Messages	Notifications
Showing rows: 1 to 12 Page No: 1			
	QUERY PLAN text		
1	Bitmap Heap Scan on sensor_data_p_202312 sensor_data_p (cost=638.93..7277.65 rows=26781 width=32) (actual time=5.296..42.587 rows=27638.00 loop...		
2	Recheck Cond: ((reading_time >= '2023-12-31 00:00:00':timestamp without time zone) AND (reading_time < '2024-01-01 00:00:00':timestamp without time ...		
3	Heap Blocks: exact=6169		
4	Buffers: shared hit=297 read=5965		
5	-> Bitmap Index Scan on sensor_data_p_202312_reading_time_idx (cost=0.00..632.23 rows=26781 width=0) (actual time=4.023..4.024 rows=27638.00 loo...		
6	Index Cond: ((reading_time >= '2023-12-31 00:00:00':timestamp without time zone) AND (reading_time < '2024-01-01 00:00:00':timestamp without time ...		
7	Index Searches: 1		
8	Buffers: shared hit=29 read=64		
9	Planning:		
10	Buffers: shared hit=53		
11	Planning Time: 1.112 ms		
12	Execution Time: 44.413 ms		
Total rows: 12		Query complete 00:00:00.159	

کوثری ۲: محاسبه میانگین روزانه دما برای یک ماه (مارس ۲۰۲۳)

- زمان اجرا : ~1196 میلی ثانیه (حدود ۱.۲ ثانیه)
- بلاک‌های خوانده شده از دیسک: 6,227
- تحلیل: سرعت اجرا حدود ۲ برابر بهبود یافت. باز هم به لطف Partition Pruning ، عملیات سنگین فقط روی پارتیشن کوچک ماه مارس انجام شد که منجر به کاهش بیش از ۱۲ برابری I/O دیسک گردید.

Data Output		Messages	Notifications
Showing rows: 1 to 11 Page No:			
QUERY PLAN text			
1	HashAggregate (cost=27466.88..28066.88 rows=40000 width=20) (actual time=1193.935..1195.365 rows=3131.00 loops=1)		
2	Group Key: sensor_data_p.device_id, date_trunc('day'::text, sensor_data_p.reading_time)		
3	Batches: 1 Memory Usage: 1041kB		
4	Buffers: shared hit=16 read=6227		
5	-> Seq Scan on sensor_data_p_202303 sensor_data_p (cost=0.00..21099.71 rows=848955 width=20) (actual time=0.595..527.739 rows=848955.00 l...		
6	Filter: ((reading_time >= '2023-03-01 00:00:00'::timestamp without time zone) AND (reading_time < '2023-04-01 00:00:00'::timestamp without time ...		
7	Buffers: shared hit=16 read=6227		
8	Planning:		
9	Buffers: shared hit=53 read=2 dirtied=1		
10	Planning Time: 2.124 ms		
11	Execution Time: 1196.088 ms		
Total rows: 11		Query complete 00:00:01.265	

کوئری ۳: خواندن داده‌های یک دستگاه خاص در یک ماه (دستگاه ۴۲، ماه مارس)

- زمان اجرا: ~26 میلی ثانیه
- بلاک‌های خوانده شده از دیسک: 41
- تحلیل: این کوئری بهبود شگفت‌انگیز ۵۹ برابری در سرعت داشت. این نتیجه حاصل ترکیب دو تکنیک قدرتمند بود: Partition Pruning و استفاده از ایندکس ترکیبی محلی روی پارتیشن ماه مارس که باعث کاهش ۱۸۶۰ برابری I/O دیسک شد.

The screenshot shows the pgAdmin 4 interface with a query plan for the following query:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM sensor_data_p
```

The query plan consists of 12 rows of information:

- 1 Bitmap Heap Scan on sensor_data_p_202303 sensor_data_p (cost=272.69..6992.78 rows=8178 width=32) (actual time=2.499..25.694 rows=8495.00 loops=1)
- 2 Recheck Cond: ((device_id = 42) AND (reading_time >= '2023-03-01 00:00:00':timestamp without time zone) AND (reading_time < '2023-04-01 00:00:00':timestamp without time zone))
- 3 Heap Blocks: exact=4629
- 4 Buffers: shared hit=88 read=4587
- 5 -> Bitmap Index Scan on sensor_data_p_202303_device_id_reading_time_idx (cost=0.00..270.65 rows=8178 width=0) (actual time=1.498..1.499 rows=8495.00 loops=1)
- 6 Index Cond: ((device_id = 42) AND (reading_time >= '2023-03-01 00:00:00':timestamp without time zone) AND (reading_time < '2023-04-01 00:00:00':timestamp without time zone))
- 7 Index Searches: 1
- 8 Buffers: shared hit=5 read=41
- 9 Planning:
- 10 Buffers: shared hit=14 dirtied=1
- 11 Planning Time: 0.508 ms
- 12 Execution Time: 26.183 ms

Summary statistics at the bottom:

- Total rows: 12
- Query complete 00:00:00.112
- CRLF

جدول مقایسه نهایی و نتیجه گیری

کوئری	زمان اجرا (تک جدولی)	زمان اجرا (پارتیشن شده)	بهبود عملکرد
۱. خواندن داده‌های یک روز	~۱۷۴۸ میلی ثانیه	~۴۴ میلی ثانیه	~۴۰ برابر سریع‌تر
۲. میانگین‌گیری یک ماه	~۲۱۹۹ میلی ثانیه	~۱۱۹۶ میلی ثانیه	~۲ برابر سریع‌تر
۳. یک دستگاه، یک ماه	~۱۵۳۸ میلی ثانیه	~۲۶ میلی ثانیه	~۵۹ برابر سریع‌تر

نتیجه‌گیری: نتایج این کارگاه به وضوح نشان می‌دهد که پارتیشن‌بندی یک استراتژی ضروری و بسیار مؤثر برای مدیریت جداول بزرگ سری-زمانی در PostgreSQL است. این تکنیک با محدود کردن اسکن داده‌ها به پارتیشن‌های مرتبط، هزینه‌های I/O را به شدت کاهش داده و عملکرد کوئری‌ها را به طرز چشمگیری بهبود می‌بخشد.

پاسخ به سوالات تئوری

سوال ۱: چه اتفاقی برای عملکرد درج (Insert) می افتد وقتی جدول پایه از اندازه حافظه (Memory) بزرگتر می شود؟

- عملکرد درج به شدت کند می شود. دلیل اصلی این است که ایندکس B-Tree جدول بسیار بزرگ شده و دیگر در حافظه (shared buffers) RAM جا نمی شود. در نتیجه، برای هر INSERT جدید، پایگاه داده مجبور است صفحات ایندکس را از دیسک بخواند، آن ها را به روزرسانی کند (که ممکن است باعث عملیات سنگین page split شود) و دوباره روی دیسک بنویسد. این حجم بالای I/O دیسک و رقابت برای قفل کردن صفحات، گلوگاه اصلی عملکرد خواهد بود.

سوال ۲: در چه زمانی ایندکس BRIN از B-Tree برای داده های سری-زمانی بهتر عمل می کند؟

- ایندکس BRIN (Block Range Index) برای داده هایی که به صورت فیزیکی روی دیسک مرتب هستند (مانند داده های سری-زمانی که معمولاً به ترتیب زمان درج می شوند) ایده آل است. BRIN حجم بسیار کمتری نسبت به B-Tree دارد، زیرا فقط حداقل و حداکثر مقدار داده در یک محدوده بزرگ از بلاک ها را ذخیره می کند.
- BRIN زمانی بهتر عمل می کند که: کوئری ها بازه های زمانی بزرگی را اسکن می کنند. در این حالت، BRIN می تواند به سرعت و با هزینه کم، محدوده های بزرگی از بلاک های دیسک را که قطعاً شامل داده های مورد نظر نیستند، نادیده بگیرد. اما برای جستجوهای بسیار دقیق و نقطه ای (Point Queries)، B-Tree همچنان سریع تر است.

سوال ۳: پارتیشن بندی چگونه به سیاست های نگهداری داده (Data Retention Policies) کمک می کند؟

- پارتیشن‌بندی این کار را فوق‌العاده آسان و سریع می‌کند. فرض کنید سیاست شما حذف داده‌های قدیمی‌تر از یک سال است.
 - بدون پارتیشن‌بندی: شما باید یک دستور DELETE سنگین اجرا کنید که میلیون‌ها ردیف را یکی‌یکی اسکن و حذف کند. این عملیات بسیار کند است و فشار زیادی به پایگاه داده وارد می‌کند.
 - با پارتیشن‌بندی: شما به سادگی می‌توانید پارتیشن‌های قدیمی را با دستور DROP PARTITION حذف کنید. این عملیات تقریباً فوری است، زیرا فقط متادیتای جدول را تغییر می‌دهد و نیازی به اسکن ردیف به ردیف ندارد. این روش بسیار کارآمدتر و سریع‌تر است.